

C  puto Concurrente 2026

Pr  ctica 1

Repaso de *Java* e

Introducci  n a la programaci  n Multihilos

Profa: Gilde Valeria Rodr  guez

1 Contexto

“La necesidad de sincronizaci  n surge siempre que existan operaciones que se ejecutan de forma concurrente sin importar si realmente se ejecutan en paralelo” [MLS24].

Esta observaci  n data de los a  os 60’s por Edsger Dijkstra, a pesar de que las arquitecturas *multiprocesador* surgen a inicios de los 2000’s, las arquitecturas *multihilo* surgen antes.

En una **arquitectura multihilo** un solo procesador tiene la capacidad de seguir m  ltiples programas de ejecuci  n distintos sin la necesidad de la intervenci  n del software, ya que esto requerir  a m  ltiples ciclos del procesador. En cambio, permite cambiar de un programa de ejecuci  n o hilo a otro a partir del almacenamiento de los estados de los hilos en el hardware, esto potencia el uso m  s eficiente de recursos del procesador, como se observa en la Figura 1.

Las primeras m  quinas con arquitecturas de multihilos surgieron en los a  os 50’s, la DYSEAC y la SEAC [MN13].

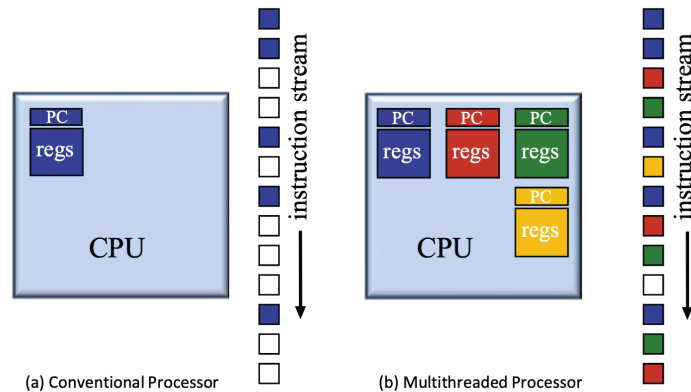


Figure 1.1: A conventional processor compared with a multithreaded processor.

Figure 1: Imagen obtenida de [MN13]

Una **arquitectura multiprocesador** est   conformada por m  ltiples procesadores, permite ejecutar dos hilos o m  s en el mismo ciclo, es decir, de forma paralela debido a que los recursos de ejecuci  n de cada hilo est  n distribuidos en los procesadores, y los procesadores a su vez tienen m  ltiples *cores*. En la Figura 2 se muestra un ejemplo de una arquitectura moderna, en donde cada procesador tiene m  ltiples cores, los cuales comparten determinadas instancias de memoria (Cach   L1, L2, L3, etc) y se comunican por un bus de memoria unificada. En la actualidad cada *core* tiene la capacidad de almacenar m  s de un hilo [Int24]. Cada arquitectura es distinta dependiendo el fabricante.

Los **hilos/procesos** se utilizan de forma diferente seg  n distintos autores, por ejemplo, seg  n

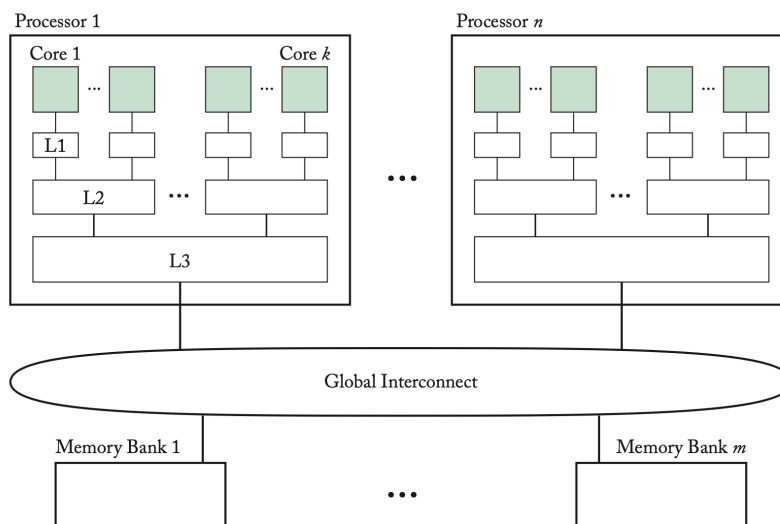


Figure 2.1: Typical symmetric (uniform memory access—UMA) machine. Numbers of components of various kinds, and degree of sharing at various levels, differs across manufacturers and models.

Figure 2: Imagen obtenida de [MLS24]

Linus Torvalds un hilo/proceso es simplemente un *contexto de ejecución*. Sin embargo, en el curso consideraremos las definiciones a continuación:

Un **proceso** es un programa en ejecución. Un **hilo** es un proceso que puede ejecutarse de forma paralela con otros hilos concurrentes y compartir recursos (por ejemplo: variables).

Un proceso también se puede considerar como un conjunto de hilos que comparten el espacio del proceso (memoria, disco, cpu, etc).

1.1 Multihilos en Java

Arquitectura de *JVM*.

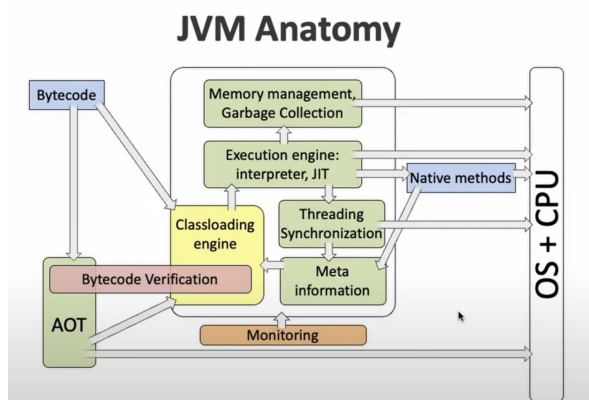


Figure 3: Imagen obtenida de [Lip]

En la Figura 4 se muestra la anatomía de la JVM, sin embargo, en este curso nuestras áreas de interés son: *Thread Synchronization* y *Memory Management*.

Un hilo en Java se implementa por un hilo nativo del sistema operativo, cada hilo tiene una región de memoria reservada la cual se denota como la *stack* de cada hilo. Cada *stack* contiene variables locales y configuraciones de métodos que son ejecutados por el hilo (El tamaño de las stacks es una variable en la JVM).

En relación con la asignación de memoria en Java, cada objeto nuevo creado con el operador *new* reside en la memoria llamada *Java heap*, la cual está especificada por la JVM (y no por los programadores). Sin embargo, esta *heap* debe ser rápida y *thread-safe*, es por eso que cada hilo tiene su propia región en esta heap.

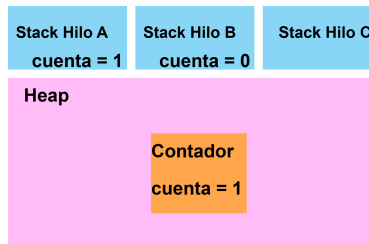


Figure 4: Representación de la Memoria en Java

Hilos en Java.

Un hilo en Java es una instancia de la clase *java.lang.Thread* (<https://docs.oracle.com/javase/8/docs/api/java/lang/Thread.html>), se crean como cualquier otro objeto en Java. Podemos crear un hilo de dos formas distintas:

1. Extendiendo la clase *Thread* y sobrescribiendo el método *run()*
2. Implementando la interfaz *Runnable* para pasarla como argumento de un objeto de la clase *Thread*.

Un hilo de la clase *Thread* tiene los siguientes atributos:

- ◊ ID
- ◊ Nombre
- ◊ Prioridad
- ◊ Status: Consta de 5 estados.

Los 5 estados son los siguientes:

1. *New*: El hilo se ha creado pero no ha empezado
2. *Runnable*: El hilo está siendo ejecutado por la JVM
3. *Blocked*: El hilo está bloqueado y está esperando por un monitor (*sleep()*, *await()*)
4. *Timed_waiting*: Un hilo está esperando a otro por un lapso de tiempo específico
5. *Terminated*: Un hilo ha terminado su ejecución

2 Ejemplos

En el siguiente link: https://github.com/surindt/FC_CConcurrente/tree/main/Programas_P1

- Programa 1 (ExampleThreads): Creacion de un hilo
- Programa 2 (ExampleMultipleExtends): Ejemplo de un contador extendiendo la clase Thread
- Programa 3 (ExampleMultipleRunnable): Ejemplo de un contador implementando la interfaz Runnable
- Programa 4 (MainThread): Ejemplo de un contador que no termina
- Programa 5 (AlwaysDesignToStop): Ejemplo de un contador que termina solo si el hilo principal lo detiene
- Programa 6 (UseJoin): Uso de Join()
- Programa 7 (ExampleMultipleExtends2): Ejemplo de un contador extendiendo la clase Thread, compartiendo un contador
- Programa 8 (ExampleMultipleRunnable2): Ejemplo de un contador implementando Runnable, compartiendo un contador
- Programa 9 (FibonacciThreads): Programa no muy paralelizable
- Programa 10 (DeterminanteConcurrente): Programa para obtener el determinante de una matriz de 3x3

3 Ejercicios

Instrucciones:

- Entrega en un **PDF** las respuestas de los ejercicios que no requieran implementarse, en los ejercicios que requieran implementación añade una breve descripción de los programas que entregas (sus nombres y qué hacen).
- Solo un integrante del equipo debe subir la práctica. Los demás integrantes deben marcar la tarea como entregada y escribir, en un comentario privado dentro de la práctica, el nombre completo de la persona que realizó la entrega.
- Cada ejercicio debe tener un programa que indique “Implementar” si aplica.
- El formato y el medio de entrega los indicará tu ayudante de laboratorio.
- Recuerda que debes utilizar **Java 21 LTS**.
- **Si no compila utilizando Java 21 LTS o si no se entrega la breve descripción de los programas se penalizará.**

Tiempo de elaboración: ≈ 1.5hr

Total de puntos: 100

1. Lee lo siguiente <https://www.evanjones.ca/software/threading-linus-msg.html> y comparte en máximo 4 líneas de computadora a que se refiere Linus Torvalds con *un contexto de ejecución* y cómo se relaciona con la definición en la sección 1 de esta práctica.
2. ¿Cuántos hilos tiene disponibles tu computadora?
Ejecuta `Runtime.getRuntime().availableProcessors()`, si son más de uno en el equipo escriban el de cada uno.
3. Revisa el programa *Determinante concurrente* y responde ¿Cuánto tiempo tarda en ejecutarse?
4. El programa *Determinante concurrente* está implementado extendiendo la clase *Thread*. Implementa el programa utilizando la interfaz *Runnable*.
5. Implementa el programa *Determinante concurrente* de forma secuencial.
6. Implementa del programa *Determinante concurrente* para dos hilos (en vez de seis).
7. Compara las 3 implementaciones: el programa *Determinante concurrente* para dos hilos, para seis hilos y el programa secuencial. Responde: ¿A qué se debe el orden en el que se ordenan los tiempos de ejecución de cada programa?
8. Si utilizas la Ley de Amdahl entre el programa *Determinante concurrente* para dos hilos y el programa secuencial. ¿El resultado es mayor o menor a 1? ¿Por qué?
9. Describe con tus propias palabras en máximo dos líneas para qué sirve el método `join()`. Si no utilizas el método `join()` en *Determinante Concurrente*, ¿sigue funcionando?

References

- [Int24] Intel. What is hyper-threading? <https://www.intel.com/content/www/us/en/gaming/resources/hyper-threading.html#:~:text=Intel%20Hyper%2DThreading%20Technology%20is%20a%20hardware%20innovation%20that,can%20be%20done%20in%20parallel.>, 2024.
- [Lip] Nikita Lipsky. Jvm anatomy 101. https://youtu.be/BeMi8K0AFAc?si=X56D_vzpp-u7iNH9.

- [MLS24] Trevor Brown Michael L. Scott. *Shared-Memory Synchronization*. Springer Cham, 30 January 2024.
- [MN13] Dean M. Tullsen Mario Nemirovsky. *Multithreading Architecture*. Springer Cham, 17 January 2013.